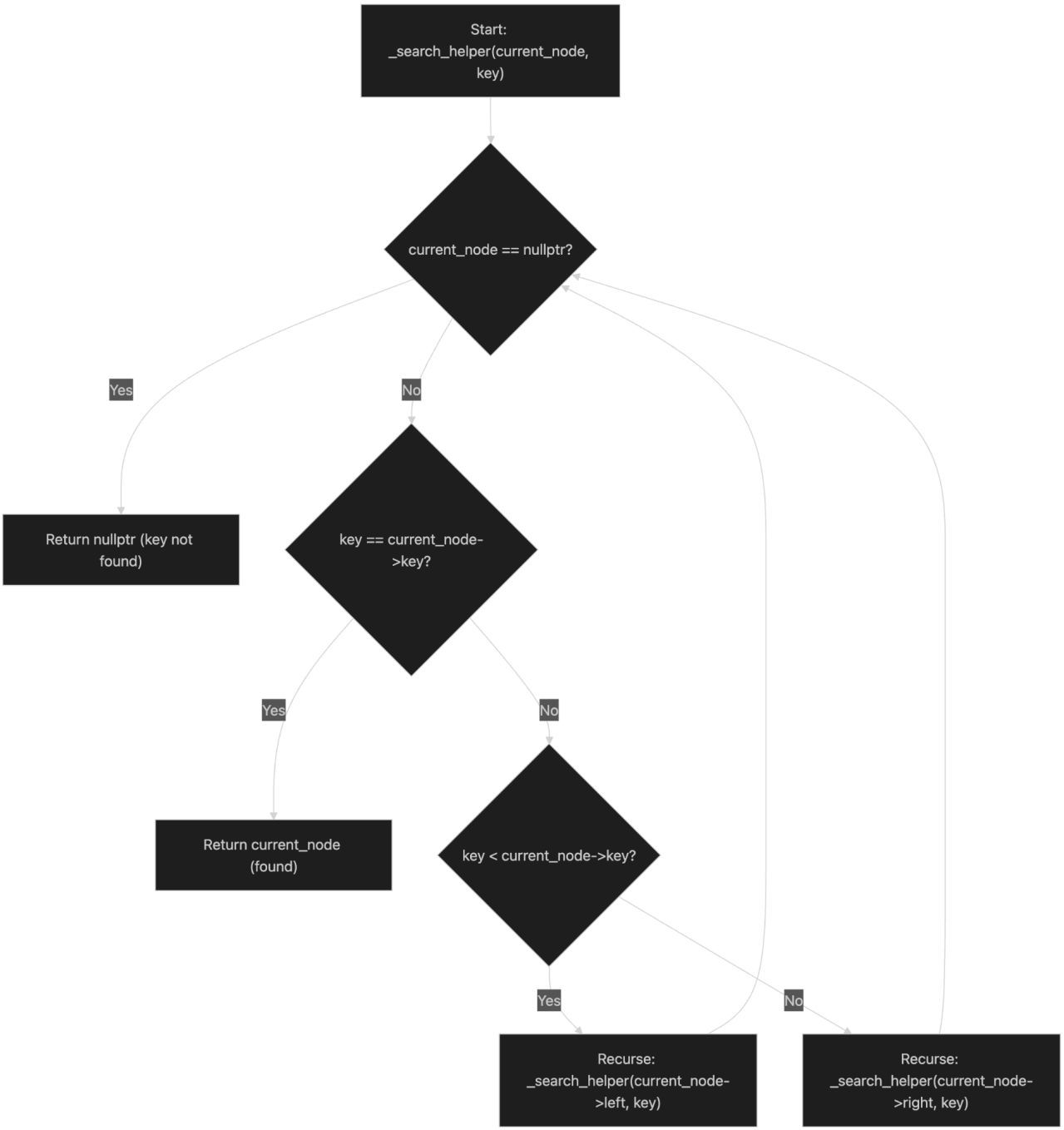
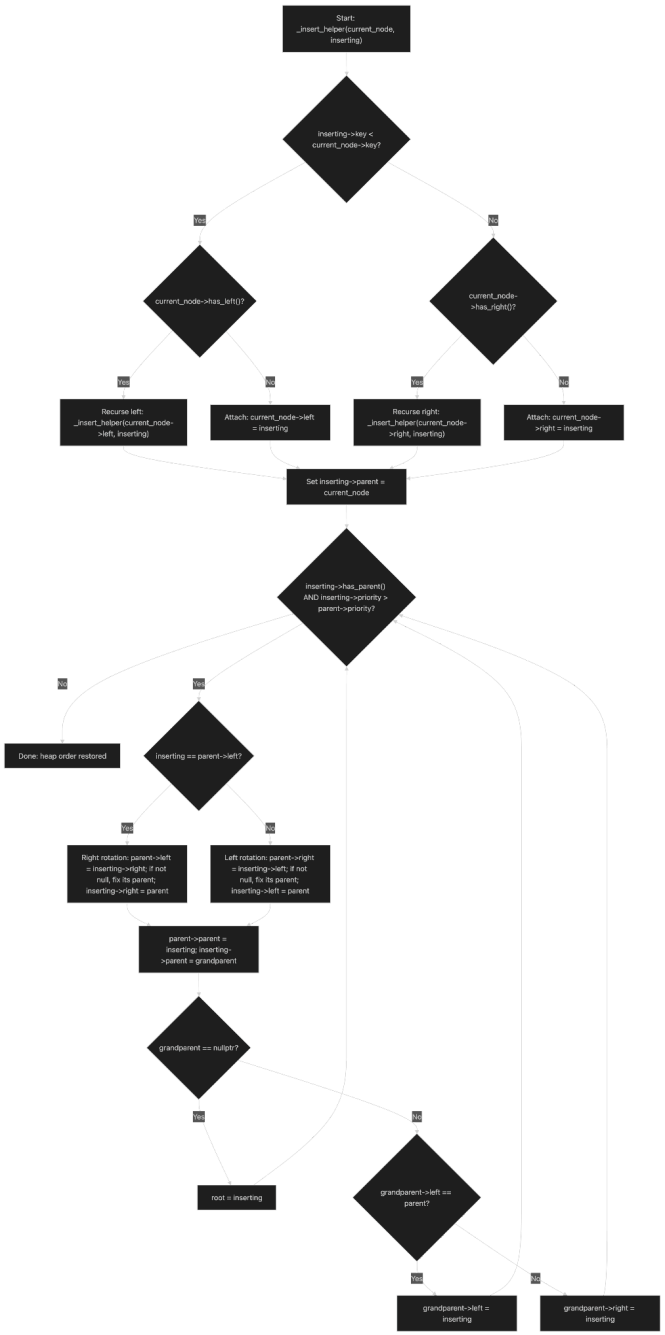
HD2 Flow Diagrams

Diagram 1: _search_helper — Standard BST Traversal



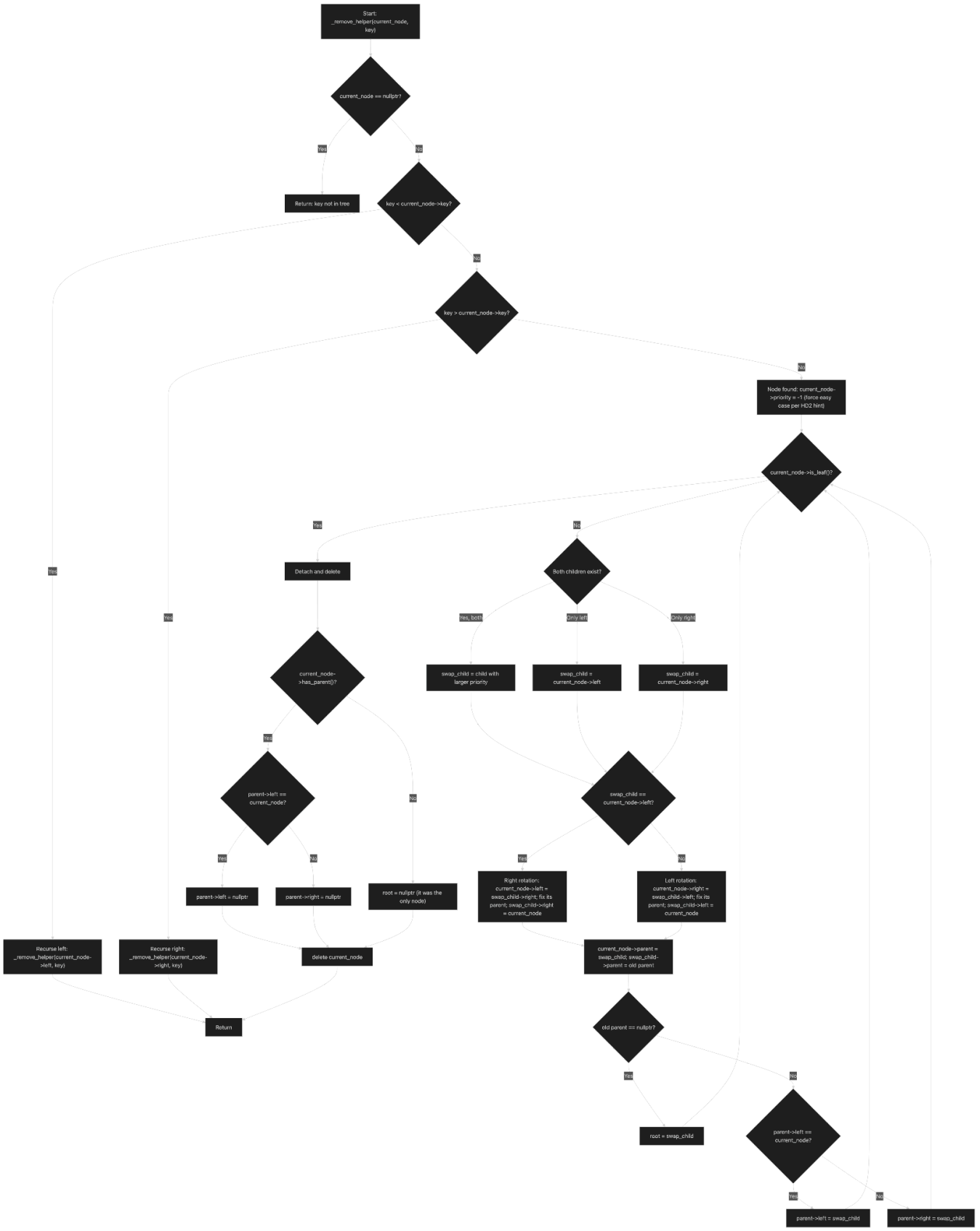
Explanation: Pure BST traversal. At each node, the key is compared three ways: equal means found, smaller means go left, larger means go right. No rotations ever happen here because search does not change the tree structure, and the relative order and priority of nodes stay untouched. Complexity is O(h), one comparison and one step down per level.

Diagram 2: _insert_helper — BST Leaf Insert, Then Inline Rotate-Up



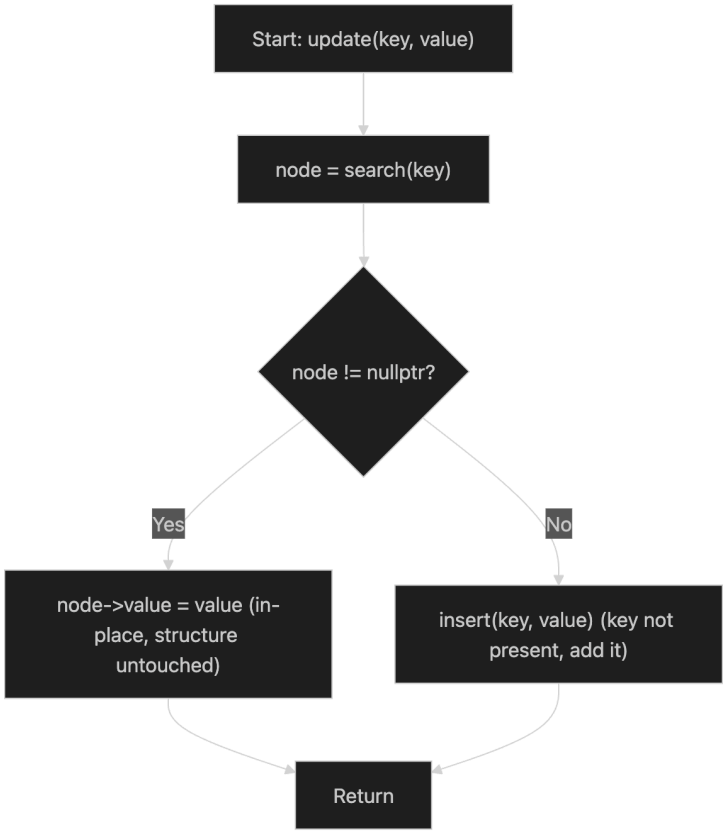
Explanation: 2 phases exactly as the HD2 handout prescribes. Phase 1 walks down the tree like a normal BST insert and attaches the new node as a leaf in the single position that satisfies key ordering, then sets its parent pointer. Phase 2 is an inline bubble-up loop: while the new node's priority beats its parent's, a rotation swaps the two. Each rotation rewires all three pointers (left, right, parent) for the moved node, the demoted parent, and the grandparent; if there is no grandparent, the inserted node becomes the new root. Rotations preserve BST key order, so after the loop both invariants hold and the tree shape is the unique one forced by the priorities. Complexity is O(h): at most one rotation per level.

Diagram 3: _remove_helper — Rotate-Down to Leaf, Then Delete



Explanation: This follows the removal hint from the handout literally. First, a standard BST descent locates the target node. Once found, its priority is set to -1, the minimum, which makes it violate the max-heap order against every possible parent. The inline loop then keeps swapping it with its higher-priority child until is_leaf() returns true, exactly the "force the easy 0-child case" strategy from the document. Every swap is a full rotation that updates left, right, and parent on all affected nodes, and if the sinking node was the root, the promoted child takes over as root. When the node finally sits as a leaf, it is detached from its parent (or the tree becomes empty if it was the last node) and freed with delete. Complexity is O(h): the BST descent is at most h steps and the sink loop performs at most h rotations.

Diagram 4: update — In-Place Value Change, No Rotations



Explanation: Update reuses search to find the node by key. If it exists, only the value field is overwritten; the tree shape, keys, and priorities are all unchanged, which is why no rotations are needed here, matching the handout note that "for the update and retrieval methods, no rotations are required as the relative order/priority of nodes is unchanged". If the key does not exist, the method falls back to insert, which runs the full leaf-insert plus rotate-up path from Diagram 2. Complexity is O(h) for the hit case, and O(h) expected for the miss case since insertion into a randomly shaped BSH runs in expected logarithmic height.

Cross-Cutting Invariant

Every rotation in Diagrams 2 and 3 is a local restructuring that preserves the in-order key sequence while changing which node sits above which. That is the whole trick behind the BSH: BST placement fixes where a node can go, priority rotations fix where it must end up, and the combination of unique priorities leaves exactly one valid shape, giving expected O(logn) height and therefore expected O(logn) for all four operations.

Trace Table: main() Insert Sequence → Final Output

Notation: x(k, p) = node with value x, key k, priority p. This trace matches the actual program output you ran.

Table 1: Step-by-Step Insertion

Step	Insert	BST Leaf Placement (Phase 1)	Rotations (Phase 2)	Tree After Step
1	a (k=3, p=8)	Empty tree → a becomes root	None (root)	a(3,8)
2	b (k=11, p=6)	11 > 3 → right child of a	6 < 8 → no bubble-up	a(3,8) with right b(11,6)
3	c (k=8, p=10)	8 > 3 → go right of a; 8 < 11 → go left of b → leaf, left child of b	p=10 > b(6) → left rotation: c rises above b; p=10 > a(8) → left rotation: c rises above a → c becomes root	Root c(8,10), left a(3,8), right b(11,6)
4	d (k=7, p=5)	7 < 8 → go left of c; 7 > 3 → go right of a → leaf, right child of a	5 < 8 → no bubble-up	a gains right child d(7,5)
5	e (k=9, p=4)	9 > 8 → go right of c; 9 < 11 → go left of b → leaf, left child of b	4 < 6 → no bubble-up	b gains left child e(9,4)
6	f (k=5, p=2)	5 < 8 → go left of c; 5 > 3 → go right of a; 5 < 7	2 < 5 → no bubble-up	d gains left child f(5,2)

		→ go left of d → leaf, left child of d		

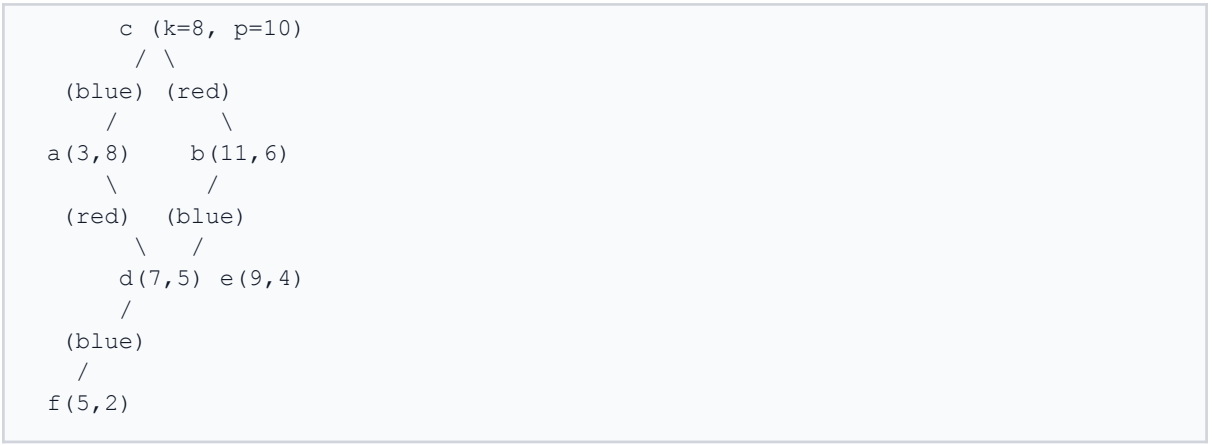
Table 2: Rotation Detail at Step 3 (the only rotation in the whole run)

Sub-step	Condition	Action	Pointers Changed	Result
3.1	c(p=10) is the left child of b(p=6); 10 > 6	Right rotation: b sinks, c rises	b->left = c->right (nullptr); c->right = b; b->parent = c; c->parent = a; a->right = c	a with right child c, and c with right child b
3.2	c(p=10) is the right child of a(p=8); 10 > 8	Left rotation: a sinks, c rises	a->right = c->left (nullptr); c->left = a; a->parent = c; c->parent = nullptr; root = c	Root c, left a, right b

Table 3: Final Verification Against Program Output

Expression in main()	Node Pointed To	Printed Output	Matches the Tree?
root	c	(c, 10)	Yes, highest priority (10)
root->left	a	(a, 8)	Yes, k=3 < 8 (left edge), p=8 < 10
root->right	b	(b, 6)	Yes, k=11 > 8 (right edge), p=6 < 10
root->left->right	d	(d, 5)	Yes, k=7 > 3 (right edge of a), p=5 < 8
root->left->right->left	f	(f, 2)	Yes, k=5 < 7 (left edge of d), p=2 < 5
root->right->left	e	(e, 4)	Yes, k=9 < 11 (left edge of b), p=4 < 6

Final Shape (identical to the figure in the HD2 handout)



Key Takeaways

1. Rotations only happen at Step 3, because c is the only node whose priority exceeds all of its ancestors. Nodes d, e, and f stop immediately at their leaf positions since their priorities are smaller than their parents'.
2. BST invariant survives the rotations: the in-order traversal of the final tree is 3(a), 5(f), 7(d), 8(c), 9(e), 11(b) — strictly ascending, even though the tree shape changed completely between Step 2 and Step 3. This is the core claim of HD2: rotations preserve key ordering.
3. Heap invariant holds on every edge: priorities decrease downward: 10→8, 10→6, 8→5, 5→2, 6→4. The root always carries the maximum priority.
4. Why f ends up at Left->Right->Left: f(k=5) is smaller than c(8) → go left; larger than a(3) → go right; smaller than d(7) → go left. Its leaf position is unique, and p=2 triggers no rotation at all.